

2026 C++

帖子: [https://blog.csdn.net/qq_33670157/article/details/104455787?](https://blog.csdn.net/qq_33670157/article/details/104455787)

ops_request_misc=%257B%2522request%255Fid%2522%253A%25224e6911a164c129ef74a968470181ae45%2522%252C%2522scm%2522%253A%252220140713.130102334..%2522%257D&request_id=4e6911a164c129ef74a968470181ae45&biz_id=0&utm_medium=distribute.pc_search_result.none-task-blog-2~all~top_positive~default-3-104455787-null-null.142^v102^control&utm_term=C%2B%2B&spm=1018.2226.3001.4187

1. 引用和指针的区别

1.指针是变量，存储的是目标变量的内存地址

引用是目标变量的别名，无独立内存空间

2.指针可以先定义后初始化，且可以为空

引用在定义时必须初始化，且不能指向空

3.指针保存的内存地址可以修改，可以指向不同的变量

引用一旦绑定不能修改

4.指针需要*解引用，&取地址

引用无需解引用

5.指针运算实际上是对保存的内存地址的计算

引用直接对目标变量进行修改

应用场景：

1.用指针的场景

(1) 硬件寄存器操作：指针可灵活指向不同寄存器地址

(2) 动态内存分配： `malloc` / `free` 返回的是指针，用于 RAM 资源动态管理 (3) 函数传参：传入实际的参数地址确保修改

(4) 中断 / 回调函数：函数指针是嵌入式中断、回调的核心

2.用引用的场景

(1) 函数参数传递大结构体：引用避免结构体拷贝，节省栈空间

(2) 类成员封装：比如驱动类中用引用绑定外设实例，保证唯一性

(3) 追求代码简洁：引用无需解引用，代码可读性更高，且无空指针风险

2. 函数重载

1. 同一个函数名对应不同的函数实现，每一类实现对应着一个函数体，名字相同，功能相同，**只是参数的类型或参数的个数不同**。

2. 多个同名函数只是函数类型（函数返回值类型）不同时，它们不是重载函数

3. inline关键字

告诉编译器：**将函数调用直接替换为函数体代码**，而非普通的函数调用（压栈 / 跳转 / 返回）

（编译阶段把函数体代码直接嵌入到调用位置，消除调用开销，提升执行速度，但会增加代码体积）

应用场景：

1. **优化高频调用的小函数**（如 GPIO 电平设置、数据校验、寄存器位操作）

2. **替代 #define 宏，提升安全性**

#define 实现简单函数（如寄存器位操作），但宏无类型检查

关键注意点：

（1）inline 是“建议”而非“强制”：编译器会根据函数体大小决定是否真的内联

（2）inline 函数的定义位置：inline 函数必须在**头文件中定义**（而非仅声明），否则编译器无法在调用处替换函数体

嵌入式中通常把 inline 函数写在 `.h` 头文件（如 `gpio.h`），方便多个 `.c/.cpp` 文件调用。

（3）嵌入式避坑：避免滥用 inline

- 中断服务函数（ISR）中调用的函数：**若函数体短且高频，可 inline**（减少中断响应时间）；但函数体大则禁止，否则中断执行时间过长，导致系统异常；
- 资源受限的 MCU（如 51 单片机）：Flash 空间小，过度 inline 会导致代码“膨胀”，甚至超出 Flash 容量，需平衡效率和体积。

4. malloc/free和new/delete的区别

1. `malloc/free` 是 C 的库函数，**仅分配 / 释放原始内存，不做初始化**

`new/delete` 是 C++ 运算符，**不仅分配内存，还会自动调用构造函数初始化对象；delete 会调用析构函数清理**

2. `malloc/free` **返回 `void*`**，需手动强制类型转换，无类型检查

`new/delete` **直接返回对应类型指针**，有编译期类型检查

3. `malloc/free` **分配失败返回 `NULL`**，需手动判空

`new/delete` **默认分配失败抛异常**

4. `malloc/free` 需手动计算数组总字节数（如 `malloc(10*sizeof(int))`）

new/delete 有专门的数组形式 `new[]/delete[]`，自动处理数组元素的构造 / 析构

5. 单片机堆区小，频繁 `malloc/free` / `new/delete` 会产生内存碎片（小块未使用内存无法合并），导致后续分配失败，嵌入式中优先用**静态缓冲区**（如 `uint8_t buf[64];`）替代动态内存

5. 类和对象

1. 构造函数可以重载，析构函数不能重载

2. this 指针

每个成员函数都有一个特殊的指针 this，它始终指向当前被调用的成员函数操作的对象

3. 以关键字 static 开头的成员为静态成员，多个类共享。

- static 成员变量属于类，不属于某个具体的对象
- 静态成员函数只能访问类中静态数据成员

4. 借助友元(friend)，可以使得其他类中得成员函数以及全局范围内得函数访问当前类得 private 成员

- 友元函数不是类的成员函数，所以没有 this 指针，必须通过参数传递对象。
- 友元函数中不能直接引用对象成员的名字，只能通过形参传递进来的对象或对象指针来引用该对象的成员。

6. 类和结构体的区别

1. 结构体成员默认公有，类成员默认私有

2. 结构体继承默认公有，类继承默认私有

3. 结构体侧重数据聚合

类侧重“面向对象”（数据 + 行为封装、继承、多态）

c 语言中的结构体不允许定义函数成员，且没有访问控制权限的属性。

4. 类的核心特性（构造 / 析构 / 虚函数）

5. c 语言中，空结构体的大小为 0，而 C++ 中空结构体大小为 1

7. C++ 特性

1. 面向对象：引入类（class）、对象、封装、继承、多态

封装：把硬件操作的“数据（寄存器地址）+ 行为（初始化 / 读写）”封装到类中，隐藏核心逻辑，避免误操作

继承 + 多态：适配嵌入式模块化驱动（如统一管理 UART、I2C、SPI 外设）

2. 更强的类型检查

`new` 运算符直接返回对应类型指针、引用：替代部分指针场景，避免野指针

8. 继承和派生

1.一个基类可以派生出多个派生类，一个派生类可以继承多个基类

2.继承方式

继承方式/基类成员	public成员	protected成员	private成员
public	public	protected	不可见
protected	protected	protected	不可见
private	private	private	不可见

利用using关键字可以改变基类成员再派生类中的访问权限；using只能修改基类中public和protected成员的访问权限

3.派生类的构造函数和析构函数

- 先执行基类的构造函数，随后执行派生类的构造函数
- 先执行派生类的析构函数，再执行基类的析构函数。

4.虚基类：引入虚基类使得派生类再继承间接共同基类时只保留一份同名成员

9. 多态和虚函数

1.不同的对象可以使用同一个函数名调用不同内容的函数。

- 静态多态性-在程序编译时系统就决定调用哪个函数，比如**函数重载**和静态多态性
- 动态多态性-在程序运行过程中动态确定调用那个函数，通过**虚函数**实现的。

2.虚函数

使用基类对象指针访问派生类对象同名函数。

- 将基类中的函数声明为虚函数，派生类中的同名函数自动为虚函数。
- 声明形式：`virtual 函数类型 函数名 (参数列表);`
- 构造函数不能声明为虚函数，析构函数可以声明为虚函数**

3.纯虚函数

在基类中不执行具体的操作，只为派生类提供统一结构的虚函数，将其声明为虚函数

抽象类：包含纯虚函数的类称为抽象类。由于纯虚函数不能被调用，所以不能利用抽象类创建对象，又称抽象基类。

10. 运算符重载

1.运算符重载是通过函数实现的，它本质上是函数重载

- 不允许重载的运算符

运算符名称	运算符
成员访问运算符	.
成员指针访问运算符	.*
域运算符	::
长度运算符	sizeof()
条件运算符	?:

2.

```
class Vector3
{
public:
    Vector3();
    Vector3(double x,double y,double z);
public:
    Vector3 operator+(const Vector3 &A)const;
    void display()const;
private:
    double m_x;
    double m_y;
    double m_z;
};
```